UNIVERSITY OF THE PEOPLE

CS 2203-01 - AY2026-T4 Programming

Assignment Activity Unit 7

Submitted by: Mubashir Ali

Instructor: John Onyianta

Date: May 21, 2026

a) Transactions: Ensuring Data Integrity and Consistency

In an online marketplace platform, transactions are fundamental mechanisms for preserving data integrity and consistency across all critical operations. A transaction is defined as a sequence of database operations treated as a single, indivisible unit of work that must either be executed in its entirety or not at all (Vidhya, Jeyaram, & Ishwarya, 2016). This all-or-nothing principle is essential in a marketplace environment where operations such as order processing, inventory management, payment handling, and user account updates are deeply interdependent.

The ACID properties, Atomicity, Consistency, Isolation, and Durability govern the behavior of transactions and form the backbone of reliable database systems (Zanini, 2023). Atomicity ensures that if any step within a transaction fails, all prior changes are automatically rolled back, preventing partial updates. Consistency guarantees that the database transitions from one valid state to another, preserving all defined rules and constraints. Isolation ensures that concurrent transactions do not interfere with one another, so two customers purchasing the same product simultaneously

will each receive a consistent and correct view of the inventory. Durability ensures that once a transaction is committed, the changes are permanent, even if the system crashes immediately afterward.

In the context of order processing, consider a customer placing a purchase. The operation involves multiple steps: verifying product availability, deducting inventory, creating an order record, and triggering payment processing. Using SQL transaction control statements, this entire sequence can be wrapped in a single transaction:

```
START TRANSACTION;

UPDATE Inventory SET stock = stock - 1 WHERE product_id = 101;

INSERT INTO Orders (user_id, product_id, status) VALUES (5, 101,
'confirmed');

UPDATE Payments SET status = 'processed' WHERE order_id = LAST_INSERT_ID();

COMMIT;
```

If any of these steps fails, for example, if the payment gateway is unavailable, a ROLLBACK statement is issued, reversing all changes and leaving the database in its original state. This prevents scenarios such as inventory being deducted without a confirmed order or payment being recorded without a corresponding order entry. By implementing transactions consistently across all critical operations, the marketplace platform ensures that users experience reliable, accurate, and tamper-resistant service at all times.

b) Static, Dynamic, and Embedded SQL in the Marketplace Platform

The online marketplace platform employs three distinct forms of SQL to handle its diverse database requirements efficiently: static SQL, dynamic SQL, and embedded SQL.

Static SQL refers to queries that are fixed and defined at the time the application is written. These queries do not change based on user input and are compiled or pre-processed before execution (Radakovic, 2017). In the marketplace context, static SQL is ideal for routine operations such as retrieving a user's order history, fetching product details by ID, or querying standard administrative

reports. Because the query structure is known in advance, static SQL benefits from precompilation, making it faster, more secure, and easier to optimize. For example, a fixed query to retrieve all active listings `SELECT * FROM Products WHERE status = 'active';` is a typical static SQL use case.

Dynamic SQL constructs queries at runtime based on user inputs or application logic (Vidhya et al., 2016). This is critical for marketplace features such as product search and filtering, where users may search by category, price range, seller rating, or keyword. Since the exact structure of these queries cannot be known in advance, dynamic SQL enables the application to build and execute them on the fly. For example, a search feature might assemble a query dynamically based on the filters a user selects. While dynamic SQL provides superior flexibility, developers must implement safeguards such as parameterized queries to prevent SQL injection attacks.

Embedded SQL integrates SQL statements directly within a host programming language such as Java or C++ (Vidhya et al., 2016). In the marketplace platform, embedded SQL can be used in the backend application layer to tightly couple database operations with business logic. For instance, an order management module written in Java can use embedded SQL to validate inventory, create records, and update payment status within the same program flow. Host variables facilitate data exchange between the application and the database, enabling seamless and efficient interaction. Together, these three SQL approaches provide the marketplace platform with a versatile toolkit for handling both predictable and complex database operations.

c) JDBC and ODBC for Database Connectivity

Establishing reliable and efficient database connections is essential for the marketplace platform to function correctly. Two standard database connectivity technologies—JDBC and ODBC—play central roles in enabling this communication.

JDBC (Java Database Connectivity) is a Java-based API that enables Java applications to connect to relational databases, execute SQL statements, and process result sets (Baeldung, n.d.). Since many enterprise-grade marketplace backends are developed using Java frameworks such as Spring Boot, JDBC provides the natural connectivity layer. The typical JDBC workflow involves loading the database driver, establishing a connection using a connection string,

creating a Statement or PreparedStatement object, executing queries, processing the ResultSet, and closing the connection. For example:

```
Connection con =
DriverManager.getConnection("jdbc:mysql://localhost:3306/marketplace",
"root", "password");

Statement stmt = con.createStatement();

ResultSet rs = stmt.executeQuery("SELECT * FROM Products WHERE
status='active'");
```

JDBC also supports transaction control, allowing developers to disable auto-commit and manually issue COMMIT or ROLLBACK commands within the application code, which is vital for the order processing workflow described in part (a). PreparedStatements in JDBC further enhance security by preventing SQL injection attacks.

ODBC (Open Database Connectivity) is a language-independent API originally developed by Microsoft that allows applications written in various programming languages, such as C, Python, or PHP, to interact with different database management systems through a standardized interface (Insightsoftware, 2023). In a marketplace platform that uses multiple backend services or administrative tools written in different languages, ODBC provides a universal connection standard without requiring language-specific drivers for each tool.

While JDBC is best suited for the Java-based core application, ODBC is valuable for cross-platform administrative dashboards, reporting tools, or data analytics components that may be built in non-Java environments. Both technologies abstract the complexity of database communication, support transaction management, and provide error handling capabilities, making them complementary components of a robust and scalable marketplace database infrastructure.

References

Baeldung. (n.d.). A guide to JDBC. Baeldung. https://www.baeldung.com/java-jdbc

Insightsoftware. (2023, June 10). What is ODBC – Open database connectivity. https://insightsoftware.com/blog/what-is-odbc/

Radakovic, M. (2017, January 13). Understanding SQL Server database static data and how it fits into database lifecycle management. SQLShack. https://www.sqlshack.com/understanding-sql-server-database-static-data-fits-database-lifecycle-management/

Vidhya, V., Jeyaram, G., & Ishwarya, K. (2016). Database management systems. Alpha Science International.

Zanini, A. (2023, February 14). Database transactions 101: The essential guide. DbVisualizer. https://www.dbvis.com/thetable/database-transactions-101-the-essential-guide/